Sorting

- Bubble Sort
- Selection Sort
- Insertion Sort
- Merge Sort
- Quick Sort
- Heap Sort
- Counting Sort
- Radix Sort

Searching

- Binary Search
- Linear Search
- Search in 2D
- Search in Rotated
- Ternary Search

DP Patterns

- 1D DP
- 2D DP
- Knapsack
- LCS
- DP on Trees
- DP on Strings
- DP on Intervals

Greedy

- Activity Selection
- Huffman Coding
- Fractional Knapsack
- Kruskal / Prim
- Minimum Spanning Tree
- Dijkstra (Greedy)
- Interval Scheduling

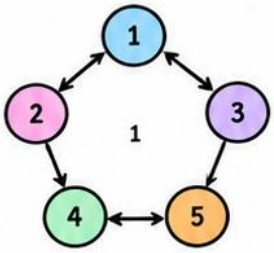
ALGORITHM CHEAT SHEET + 100 INTERVIEW QUESTIONS

Time & Space Complexity | Data Structures | Graph Algorithms | Common Templates | Pattern Based Questions

codedsoul_05

Graph Algorithms

- BFS / DFS
- Dijkstra
- Bellman Ford
- Kruskal / Prim
- Topological Sort
- SCC
- Union Find



Data Structures

- Arrays
- Linked List
- Stack / Queue
- HashMap / Set
- Tree / BST
- Heap (Priority Queue)
- Trie
- Disjoint Set Union

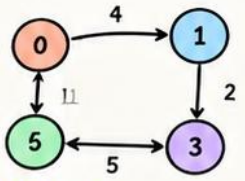
String Algorithms

- KMP
- Z Algorithm
- Rabin Karp
- Trie
- Suffix Array
- Rolling Hash

Advanced Topics

- Topological Sort
- Strongly Connected Components (SCC)
- Bridges & Articulation
- Eulerian Path / Circuit
- Minimum Cost Flow
- Network Flow
- Bipartite Matching

Dijkstra




```
dist[0] = 0
for u, v, w in edges:
  if dist[v] > dist[u] + w:
    dist[v] = dist[u] + w
```

Dynamic Programming

```
for i in range(1, n):
  for j in range(1, n):
    dp[i][j] = max(
      dp[i-1][j],
      dp[i][j-1]
    )
```

Prefix Sum

2	4	1	3	5
↓	↓	↓	↓	↓
2	6	7	10	15

codedsoul_05

Think
Solve
Optimize
Get Hired

ALGORITHMS
INTERVIEW PREP
PROBLEM SOLVING

Same Patterns Everywhere

1. Algorithm Fundamentals + Complexity Analysis

What is an Algorithm ?

- An algorithm is a step by step procedure to solve a problem.
- It takes input, performs a set of well defined steps and gives an output.
- It must be finite, unambiguous and effective.

Characteristics of an Algorithm

1. **Input** : Takes zero or more inputs
2. **Output** : Produces at least one output
3. **Definiteness** : Steps are clear and unambiguous
4. **Finiteness** : Must terminate after finite steps
5. **Effectiveness** : Each step must be simple and feasible

Types of Algorithms

Type	Description
Brute Force	Tries all possible solutions. Simple but may be slow.
Divide and Conquer	Breaks problem into smaller subproblems.
Greedy	Makes locally optimal choices.
Dynamic Programming	Solves overlapping subproblems using memoization.
Backtracking	Tries all possibilities and backtracks if needed.
Randomized	Uses randomness to solve a problem.
Approximation	Finds near optimal solution for complex problems.

Example Algorithm

Find the maximum element in an array

```
Algorithm MaxElement(arr, n)
1  max = arr[0]
2  for i = 1 to n-1
3    if arr[i] > max
4       max = arr[i]
5  return max
```

Dry Run Example

arr = [4, 2, 8, 1, 6]

i	arr[i]	max (before)	Comparison	max (after)
0	4	-	-	4
1	2	4	2 > 4 ? no	4
2	8	4	8 > 4 ? yes	8
3	1	8	1 > 8 ? no	8
4	6	8	6 > 8 ? no	8

Algorithm Design Paradigms

1. Brute Force
2. Divide and Conquer
3. Greedy
4. Dynamic Programming
5. Backtracking
6. Branch and Bound
7. Randomized Algorithms
8. Approximation Algorithms

Different techniques are used depending on the problem type.

Choosing the right paradigm helps in writing efficient algorithms.

Analyzing an Algorithm

- We analyze an algorithm to estimate the resources it uses like time and memory.
- It helps us compare different algorithms and choose the best one.
- We usually focus on the worst case, average case and best case.

Complexity Analysis

- Complexity analysis measures the amount of time and space an algorithm requires as a function of input size n.
- It helps us understand how the algorithm will perform when the input size grows.
- We use Big O, Big Omega and Big Theta notations to express complexity.

Time Complexity

- Time complexity measures how the running time of an algorithm grows with input size n.
- It is expressed using asymptotic notations.

Common Time Complexities

Notation	Name
$O(1)$	Constant Time
$O(\log n)$	Logarithmic Time
$O(n)$	Linear Time
$O(n \log n)$	Linearithmic Time
$O(n^2)$	Quadratic Time
$O(n^3)$	Cubic Time
$O(2^n)$	Exponential Time
$O(n!)$	Factorial Time

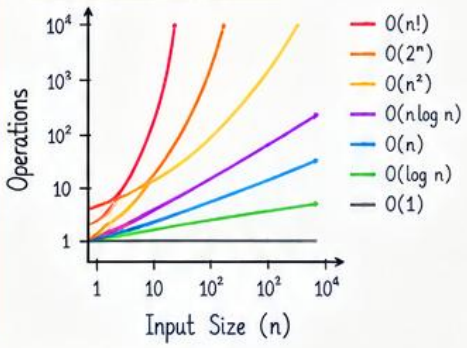
Space Complexity

- Space complexity measures the amount of memory an algorithm uses with respect to input size n.
- It includes auxiliary space used for variables, data structures and recursive calls.

Common Space Complexities

Notation	Name
$O(1)$	Constant Space
$O(\log n)$	Logarithmic Space
$O(n)$	Linear Space
$O(n \log n)$	Linearithmic Space
$O(n^2)$	Quadratic Space

Growth Comparison



Key Takeaways

- ✅ An algorithm is a finite step by step solution to a problem.
- ✅ We analyze algorithms using time and space complexity.
- ✅ Different problems require different algorithm design paradigms.
- ✅ Complexity helps us choose the most efficient algorithm.

2. Time Complexity + Space Complexity

What is Time Complexity ?

- Time complexity measures how the running time of an algorithm grows with input size n.
- It helps us estimate the number of basic operations executed by the algorithm.
- We use asymptotic notation to express the growth rate, ignoring constants and lower order terms.

What is Space Complexity ?

- Space complexity measures the amount of memory an algorithm uses with respect to input size n.
- It includes auxiliary space (extra space) used for variables, data structures and recursive calls.
- We usually ignore input space while calculating auxiliary space.

Example : Time Complexity

```
for i = 0 to n - 1
    print(i)
```

This loop runs n times.

Time Complexity : O(n)

Example : Space Complexity

```
int[] arr = new int[n]
```

We are using an array of size n.
Extra space used = n.

Space Complexity : O(n)

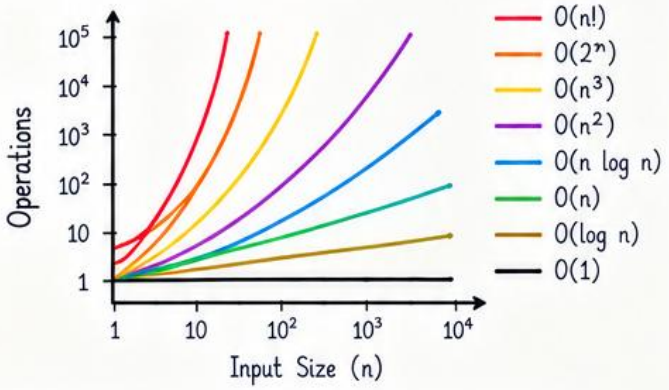
Common Time Complexities

Notation	Name	Growth Rate	Example Use Case
O(1)	Constant Time	Does not grow	Accessing array element
O(log n)	Logarithmic Time	Very slow growth	Binary search
O(n)	Linear Time	Linear growth	Single loop
O(n log n)	Linearithmic Time	Moderate growth	Merge sort, Heap sort
$O(n^2)$	Quadratic Time	Fast growth	Nested loops
$O(n^3)$	Cubic Time	Very fast growth	Triple nested loops
$O(2^n)$	Exponential Time	Extremely fast	Recursion (Fibonacci)
O(n!)	Factorial Time	Extremely fast	Generating permutations

Common Space Complexities

Notation	Name	Example Use Case
O(1)	Constant Space	Few variables
O(log n)	Logarithmic Space	Binary search (recursive)
O(n)	Linear Space	Array, hash map, recursion stack
O(n log n)	Linearithmic Space	Merge sort (array + recursion)
$O(n^2)$	Quadratic Space	2D matrix, adjacency matrix
$O(2^n)$	Exponential Space	Recursion with overlapping states
O(n!)	Factorial Space	Storing all permutations

Growth Rate Comparison



Analyzing Time Complexity (Examples)

Code	Analysis	Time Complexity
`arr[i]`	Single operation	O(1)
`for i = 0 to n-1`	Runs n times	O(n)
`for i = 0 to n-1` `  for j = 0 to n-1`	Runs n × n times	$O(n^2)$
`for i = 0 to n-1` `  for j = i to n-1`	Runs n(n + 1)/2 times	$O(n^2)$
`while (n > 1)` `  n = n / 2`	n is divided by 2 each time	O(log n)

`for i = 1 to n`
`  for j = 1 to n`
`  for k = 1 to n`
	Runs n × n × n times	$O(n^3)$

Analyzing Space Complexity (Examples)

Code	Analysis	Space Complexity
`int x`	Single variable	O(1)
`int[] arr = new int[n]`	Array of size n	O(n)
`HashMap<Character, Integer> map`	Stores up to n elements	O(n)

`void solve(int n)`
`{`
`  if (n == 0) return;`
`  solve(n - 1);`
`}`
	Recursive call stack of size n	O(n)

MergeSort (array)	Auxiliary array of size n + recursion stack	O(n)
Binary Search (recursive)	Recursion stack of size log n	O(log n)

Key Points to Remember

1. We focus on the worst case complexity in interviews.
2. Ignore constant factors and lower order terms.
3. Time complexity shows running time, space complexity shows memory usage.
4. Use Big O notation to express the growth rate.
5. Different algorithms can solve the same problem with different complexities.
6. Always try to optimize both time and space when possible.
7. Space complexity includes auxiliary space and recursion stack.
8. Choose the right data structure and algorithm for better complexity.

3. Searching Algorithms + Linear Search

What is Searching ?

- Searching is the process of finding an element in a collection of data.
- We are given a **target** value and we need to check whether it exists in the data structure and return its position (index) if found.
- Searching algorithms are fundamental and used in many real world applications like search in contacts, search in databases, etc.

Types of Searching Algorithms



Uninformed Search (No order)

1. Linear Search
2. Jump Search
3. Interpolation Search

Informed Search (Requires sorted data)

1. Binary Search
2. Ternary Search
3. Exponential Search
4. Fibonacci Search

Linear Search

- Linear **Search** is the simplest searching algorithm.
- It checks **each** element one by one from the beginning until the target element is found.
- It works on both sorted and unsorted data.
- Time complexity is O(n) which means in worst case we may have to check all elements.

How Linear Search Works ?

1. Start from the first element.
2. Compare the current element with the target.
3. If equal, return the index.
4. If not equal, move to the next element.
5. Repeat until the end of the array.
6. If the element is not found, return -1.

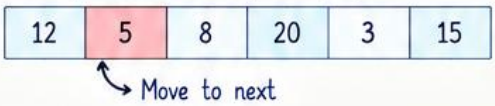
Visual Example

Array: [12 | 5 | 8 | 20 | 3 | 15] Target = 20

Step 1: Compare 12 with 20 (not equal)



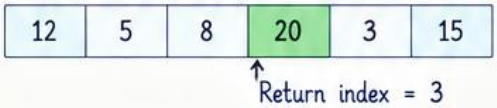
Step 2: Compare 5 with 20 (not equal)



Step 3: Compare 8 with 20 (not equal)



Step 4: Compare 20 with 20 (found)



Algorithm (Pseudocode)

```
1.  Start
2.  for i = 0 to n - 1
3.      if arr[i] == target
4.          return i
5.  return -1
6.  End
```

Code Example

Python

```python
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

# Example
arr = [12, 5, 8, 20, 3, 15]
target = 20
index = linear_search(arr, target)
print("Element found at index:", index)
```

Complexity Analysis

Case	Time Complexity	Space Complexity	Explenation
Best Case	O(1)	O(1)	Element is at first position
Average Case	O(n)	O(1)	Element is in middle
Worst Case	O(n)	O(1)	Element is at last position or not present at all

Pros and Cons

Advantages

- ✅ Very simple to implement
- ✅ Works on both sorted and unsorted data
- ✅ Does not require extra space
- ✅ Useful for small datasets

Disadvantages

- ❌ Time complexity is O(n) which is slow for large datasets
- ❌ Not efficient for large data
- ❌ Checks each element one by one

4. Binary Search + Variations

What is Binary Search ?

- Binary Search is a searching algorithm used to find the position of a target element in a sorted array.
- It repeatedly divides the search space into half.
- It compares the middle element with the target and decides whether to search in the left half or right half.
- It works only on sorted data.
- Time complexity is O(log n) which is much better than linear search.

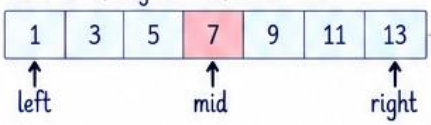
How Binary Search Works ?

1. Find the middle index mid = (left + right) / 2
2. Compare arr[mid] with target
3. If equal, return mid
4. If target < arr[mid], search in left half
5. If target > arr[mid], search in right half
6. Repeat until left > right

Visual Example

Array: [1 | 3 | 5 | 7 | 9 | 11 | 13] Target = 7

1 left = 0, right = 6, mid = 3

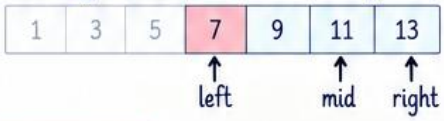


arr[mid] = 7 equals target
Return 3

2 If target < arr[mid] (search left half)



3 If target > arr[mid] (search right half)



Pseudocode

```
1.  left = 0, right = n - 1
2.  while left <= right:
3.        mid = (left + right) / 2
4.        if arr[mid] == target:
5.             return mid
6.        elif arr[mid] < target:
7.             right = mid - 1
8.        else:
9.             left = mid + 1
10. return -1
```

Key Takeaways

- Binary Search works only on sorted data.
- Time complexity is O(log n).
- Can be modified for many real world problems.
- Used in interviews frequently.
- Important variations include lower bound, upper bound, rotated array and binary search on answer.

Code Example

Pythom

```python
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            right = mid - 1
        else:
            left = mid + 1
    return -1

# Example
arr = [1, 3, 5, 7, 9, 11, 13]
target = 7
print(binary_search(arr, target))  # Output: 3
```

Complexity Analysis

Case	Time Complexity	Space Complexity	Explanation
Best Case	O(1)	O(1)	Target is at middle
Average Case	O(log n)	O(1)	Search space reduces by half
Worst Case	O(log n)	O(1)	Target not present or at last step

Binary Search Variations

1	Search in Rotated Sorted Array	Array is sorted but rotated at some pivot. Example: [4, 5, 6, 7, 0, 1, 2]
2	Find First Occurrence (Lower Bound)	Find the index of first occurrence of target in case of duplicates.
3	Find Last Occurrence (Upper Bound)	Find the index of last occurrence of target in case of duplicates.
4	Search Insert Position	Find the index where target can be inserted to keep the array sorted.
5	Find Minimum in Rotated Sorted Array	Find the minimum element in a rotated sorted array.
6	Find Peak Element	Find an element that is greater than its neighbors.
7	Binary Search on Answer	Used to solve optimization problems like minimum time, capacity, etc.
8	Search in 2D Matrix	Matrix is sorted row wise and column wise.

Practice Problems

1. Search in a Sorted Array
2. Find First and Last Position of Element
3. Search Insert Position
4. Search in Rotated Sorted Array
5. Find Minimum in Rotated Sorted Array
6. Find Peak Element
7. Search in 2D Matrix
3. Koko Eating Bananas (Binary Search on Answer)
9. Capacity to Ship Packages
10. Aggressive Cows

5. Sorting Algorithms + Bubble Sort

What is Sorting ?

- Sorting means arranging the elements of an array in a specific order (ascending or descending).
- It helps in faster searching, efficient data processing and better problem solving.
- Sorting algorithms are widely used in real world applications like databases, search engines, etc.

Types of Sorting Algorithms

Comparison Based Sorting

- Compare elements to decide the order.
- Examples: Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort, Heap Sort.

Non-Comparison Based Sorting

- Do not compare elements directly.
- Examples: Counting Sort, Radix Sort, Bucket Sort.

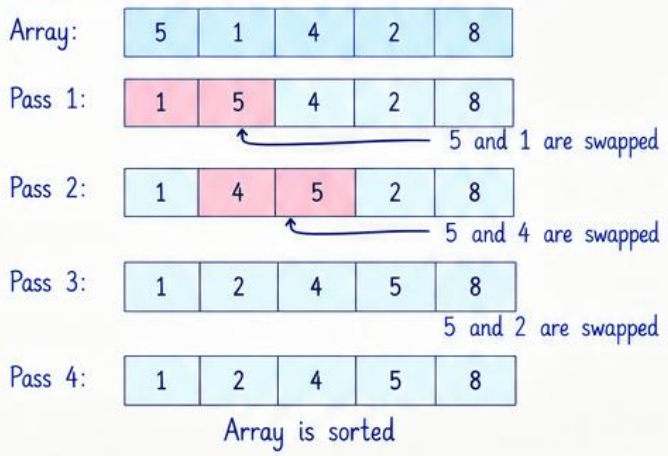
Common Sorting Algorithms

Algorithm	Best Case	Average Case	Worst Case	Space	Stable
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	No
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	No
Counting Sort	O(n + k)	O(n + k)	O(n + k)	O(k)	Yes
Radix Sort	O(nk)	O(nk)	O(nk)	O(n + k)	Yes

Introduction to Bubble Sort

- Bubble Sort is the simplest sorting algorithm.
- It repeatedly compares adjacent elements and swaps them if they are in the wrong order.
- In each pass, the largest element "bubbles up" to the end.
- It is easy to implement but not efficient for large datasets.
- Time Complexity: $O(n^2)$ in average and worst case.
- Space Complexity: O(1) (in-place sorting).

Dry Run Example (Ascending Order)



How Bubble Sort Works ?

1. Compare the first two elements.
2. If they are in the wrong order, swap them.
3. Move to the next pair and repeat.
4. After each pass, the largest element is placed at the end of the array.
5. Repeat the process for n - 1 passes.

Algorithm (Pseudocode)

```
1.  for i = 0 to n - 2
2.      for j = 0 to n - i - 2
3.          if arr[j] > arr[j + 1]
4.              swap(arr[j], arr[j + 1])
```

Code Example

Python

```python
def bubble_sort(arr);
    n = len(arr)
    for i in range(n - 1):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

# Example
arr = [5, 1, 4, 2, 8]
print("Sorted array;", bubble_sort(arr))
# Output: [1, 2, 4, 5, 8]
```

@codedsoul_05

Complexity Analysis

Case	Time Complexity	Space Complexity	Explanation
Best Case	O(n)	O(1)	Array is already sorted (no swaps)
Average Case	$O(n^2)$	O(1)	Elements are in random order
Worst Case	$O(n^2)$	O(1)	Array is sorted in reverse order

Advantages

- ✅ Very simple and easy to implement.
- ✅ In-place sorting (O(1) space).
- ✅ Good for small datasets.
- ✅ Helps in understanding basic sorting concepts.

Disadvantages

- ❌ Not efficient for large datasets.
- ❌ Time complexity is $O(n^2)$.
- ❌ Performs many unnecessary swaps.
- ❌ Not suitable for real world applications.

Key Takeaways

- ⭐ Bubble Sort is a simple comparison based sorting algorithm.
- ⭐ It repeatedly swaps adjacent elements.
- ⭐ Time Complexity is $O(n^2)$ and Space Complexity is O(1).
- ⭐ It is useful for small datasets and learning purposes.

6. Selection Sort + Insertion Sort

What is Selection Sort ?

- Selection Sort is a simple comparison based sorting algorithm.
- It divides the array into two parts: sorted and unsorted.
- In each iteration, it finds the minimum element from the unsorted part and swaps it with the first element of the unsorted part.
- It performs exactly n - 1 swaps (at most).

How Selection Sort Works ?

1. Find the minimum element from the unsorted part.
2. Swap it with the first element of the unsorted part.
3. Move the boundary of the sorted part by one.
4. Repeat for n - 1 iterations.

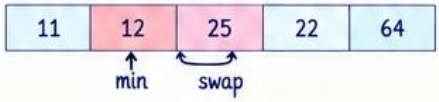
Dry Run Example (Ascending Order)

Array: 64 25 12 22 11

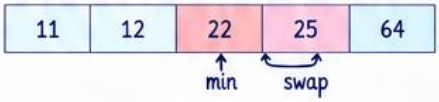
Pass 1: Find minimum (11) and swap with 64



Pass 2: Find minimum (12) and swap with 25



Pass 3: Find minimum (22) and swap with 25



Pass 4: Find minimum (25) and swap with 25 (no change)



Array is sorted

Algorithm (Pseudocode)

```
for i = 0 to n - 2
    min_idx = i
    for j = i + 1 to n - 1
        if arr[j] < arr[min_idx]
            min_idx = j
    swap(arr[i], arr[min_idx])
```

Complexity Analysis

Case	Time Complexity	Space Complexity	Explenation
Best Case	$O(n^2)$	O(1)	Always n^2 comparisons
Average Case	$O(n^2)$	O(1)	Always n^2 comparisons
Worst Case	$O(n^2)$	O(1)	Always n^2 comparisons

Advantages

- Simple to understand
- In-place sorting (O(1) space)
- Works well for small datasets

Disadvantages

- Not efficient for large datasets
- Time complexity is $O(n^2)$
- Performs many unnecessary swaps

What is Insertion Sort ?

- Insertion Sort builds the sorted array one element at a time.
- It takes the current element and inserts it into its correct position in the sorted part.
- It works similarly to the way we sort playing cards in our hands.
- It is efficient for small datasets and nearly sorted arrays.

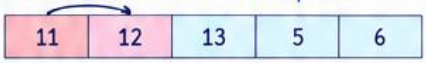
How Insertion Sort Works ?

1. Start from the second element (index 1).
2. Compare the current element with elements in the sorted part (on its left).
3. Shift elements to the right if they are greater.
4. Insert the current element at the correct position.
5. Repeat for all elements.

Dry Run Example (Ascending Order)

Array: 12 11 13 5 6

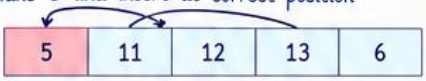
Pass 1: Take 11 and insert at correct position



Pass 2: Take 13 (already in correct position)



Pass 3: Take 5 and insert at correct position



Pass 4: Take 6 and insert at correct position



Array is sorted

Algorithm (Pseudocode)

```
for i = 1 to n - 1
    key = arr[i]
    j = i - 1
    while j >= 0 and arr[j] > key
        arr[j + 1] = arr[j]  # shift to right
        j = j - 1
    arr[j + 1] = key         # insert key
```

Complexity Analysis

Case	Time Complexity	Space Complexity	Explanation
Best Case	O(n)	O(1)	Array is already sorted
Average Case	$O(n^2)$	O(1)	Random order elements
Worst Case	$O(n^2)$	O(1)	Array is in reverse order

Advantages

- Simple to understand and implement
- Efficient for small or nearly sorted data
- In-place sorting (O(1) space)
- Stable sorting algorithm

Disadvantages

- Not efficient for large datasets
- Time complexity is $O(n^2)$
- Many element shifts in worst case

@codedsoul_05

7. Merge Sort + Divide and Conquer

What is Divide and Conquer ?

- Divide and Conquer is an algorithmic paradigm that breaks a problem into smaller subproblems, solves them recursively and combines their solutions.
- It is used in many algorithms like Merge Sort, Quick Sort, Binary Search, etc.
- It helps in solving large problems efficiently by reducing them to simpler versions of the same problem.

Steps in Divide and Conquer

Divide → Break the problem into smaller subproblems

Conquer → Solve the subproblems recursively

Combine → Merge the solutions to get the final result

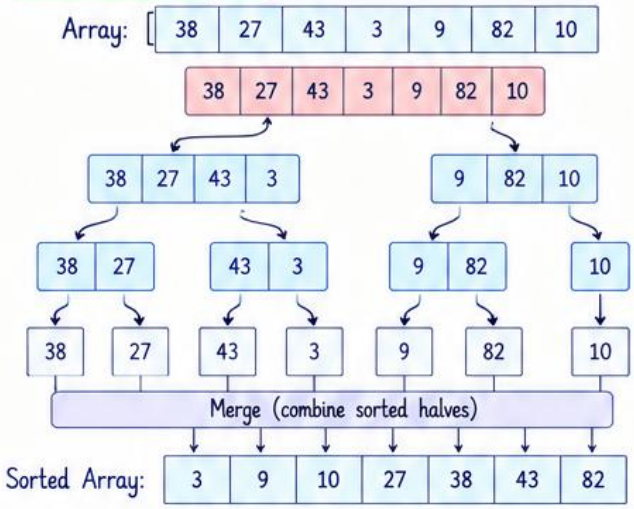
Merge Sort

- Merge Sort is a sorting algorithm based on Divide and Conquer.
- It divides the array into two halves, sorts them recursively and then merges the sorted halves.
- It is a stable sorting algorithm.
- Time Complexity: O(n log n) in all cases.
- Space Complexity: O(n) (requires extra space).

How Merge Sort Works ?

1. If the array has 0 or 1 element, it is already sorted.
2. Divide the array into two halves.
3. Recursively sort the left half.
4. Recursively sort the right half.
5. Merge the two sorted halves.

Visual Example



Advantages and Disadvantages

Advantages

- ✅ Stable sorting algorithm
- ✅ Time complexity is always O(n log n)
- ✅ Works well for large datasets
- ✅ Good for linked lists and external sorting

Disadvantages

- ❌ Requires extra space O(n)
- ❌ Not in-place sorting
- ❌ Slower than Quick Sort for small arrays

Code Implementation

Python

```python
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

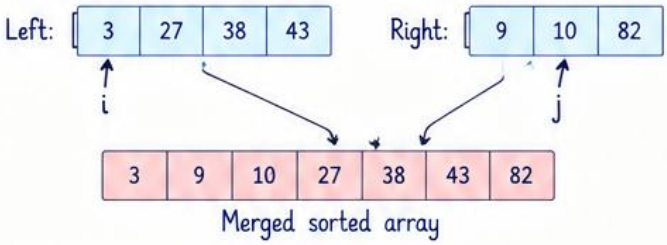
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result

# Example
arr = [38, 27, 43, 3, 9, 82, 10]
print(merge_sort(arr)) # Output: [3, 9, 10, 27, 38, 43, 82]
```

@codedsoul

Merge Function Visualization



Complexity Analysis

Case	Time Complexity	Space Complexity	Explanation
Best Case	O(n log n)	O(n)	Array is divided into halves at each step
Average Case	O(n log n)	O(n)	Same as best case
Worst Case	O(n log n)	O(n)	Same as best case

Key Takeaways

- ⭐ Merge Sort is based on Divide and Conquer.
- ⭐ It always takes O(n log n) time.
- ⭐ It is stable but not in-place.
- ⭐ It works well for large datasets and linked lists.
- ⭐ Divide and Conquer is a powerful paradigm used in many important algorithms.

8. Quick Sort + Partitioning

What is Quick Sort ?

- Quick Sort is a divide and conquer based sorting algorithm.
- It picks a pivot element, partitions the array into two parts (elements smaller and greater than pivot).
- It recursively sorts the left and right parts.
- It is an in-place sorting algorithm.
- It is one of the most widely used sorting algorithms because of its simplicity and good average performance.

Key Idea

- Choose a pivot element (can be first, last, middle or random).
- Partition the array so that:
  - Elements smaller than pivot come to the left.
  - Elements greater than pivot come to the right.
  - Pivot is placed at its correct position.
- Recursively apply the same process on left and right subarrays.

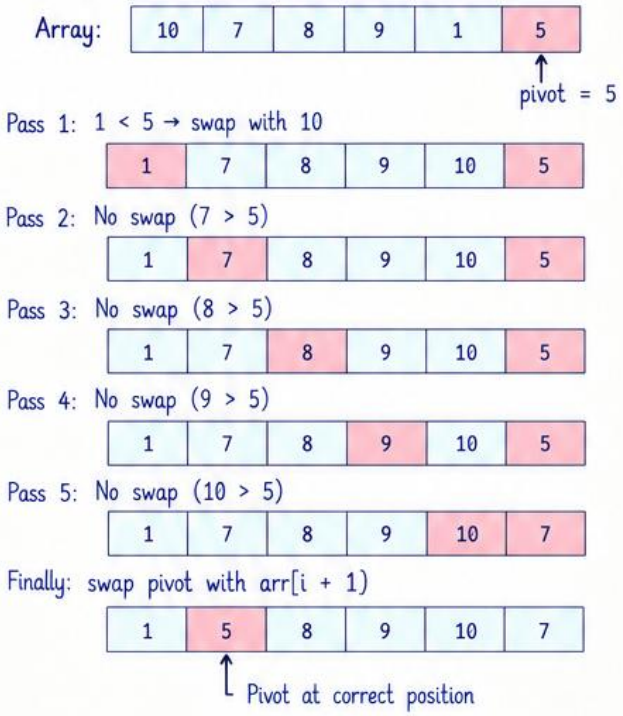
Partitioning (Lomuto Scheme)

- Partitioning rearranges elements around a pivot.
- After partitioning, pivot is at its correct sorted position.
- All elements to the left are smaller than pivot.
- All elements to the right are greater than pivot.

How Partitioning Works ?

1. Choose the last element as pivot.
2. Maintain an index i for smaller elements.
3. Traverse the array and swap elements smaller than pivot with arr[i].
4. Finally swap pivot with arr[i + 1].

Visual Example of Partitioning



Algorithm (Lomuto Partition)

```
1.  partition(arr, low, high)
2.      pivot = arr[high]
3.      i = low - 1
4.      for j = low to high - 1
5.          if arr[j] < pivot:
6.              i = i + 1
7.              swap(arr[i], arr[j])
8.      swap(arr[i + 1], arr[high])
9.      return i + 1  # pivot index
```

Quick Sort (Pseudocode)

```
1.  quick_sort(arr, low, high)
2.      if low < high:
3.          pi = partition(arr, low, high)
4.          quick_sort(arr, low, pi - 1)
5.          quick_sort(arr, pi + 1, high)
```

Code Implementation

Python

```python
def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] < pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

def quick_sort(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)

# Example
arr = [10, 7, 8, 9, 1, 5]
quick_sort(arr, 0, len(arr) - 1)
print("Sorted array:"; arr)  # Output: [1, 5, 7, 8, 9, 10]
```

@codedsoul_05

Complexity Analysis

Case	Time Complexity	Space Complexity	Explanation
Best Case	O(n log n)	O(log n)	Good pivot divides array into halves
Average Case	O(n log n)	O(log n)	Random input
Worst Case	$O(n^2)$	O(log n)	Pivot is smallest or largest (e.g., already sorted array)

Advantages

- ✅ Very fast in practice
- ✅ In-place sorting (O(log n) space)
- ✅ Good average case performance
- ✅ Widely used in real world applications

Disadvantages

- ❌ Worst case time complexity $O(n^2)$
- ❌ Performance depends on pivot choice
- ❌ Not stable (relative order not preserved)
- ❌ Can be slow on already sorted data (if bad pivot is chosen)

Key Takeaways

- ⭐ Quick Sort is based on Divide and Conquer.
- ⭐ Partitioning places the pivot at its correct position.
- ⭐ Average time complexity is O(n log n).
- ⭐ Choice of pivot is important to avoid worst case.

9. Heap Sort + Counting Sort

What is Heap Sort ?

- Heap Sort is a comparison based sorting algorithm that uses a Binary Heap (usually a Max Heap).
- It repeatedly extracts the maximum element from the heap and places it at the end of the array.
- It is an in-place sorting algorithm.
- It has a time complexity of O(n log n) in all cases.
- It is not stable.

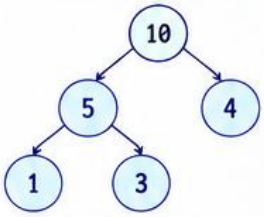
How Heap Sort Works ?

1. Build a Max Heap from the given array.
2. Swap the root (maximum element) with the last element.
3. Reduce the heap size by 1 and heapify the root.
4. Repeat until the array is sorted.

Building a Max Heap (Example)

Array:

4	10	3	5	1



Convert the array into a Max Heap (largest element at root).

Heap Sort Process (Example)

Array:

4	10	3	5	1

@codedsoul_05

1. Build Max Heap

10	5	3	4	1

2. Swap root with last element

1	5	3	4	10

10 is placed at correct position

3. Heapify the reduced heap

5	4	3	1	10

Repeat the process...

Sorted Array:

1	3	4	5	10

Algorithm (Pseudocode)

```
heap_sort(arr):
    build_max_heap(arr)
    for i = n - 1 to 1:
        swap(arr[0], arr[i])
        heapify(arr, 0, i)
```

Complexity Analysis

Case	Time Complexity	Space Complexity	Explanation
Best Case	O(n log n)	O(1)	Building heap +n extractions
Average Case	O(n log n)	O(1)	Same as best case
Worst Case	O(n log n)	O(1)	Same as best case

Advantages

- ✅ Good time complexity (O(n log n)).
- ✅ In-place sorting (O(1) space).
- ✅ Works well for large datasets.

Disadvantages

- ❌ Not stable.
- ❌ Slower than Quick Sort in practice.
- ❌ More complex to implement.

What is Counting Sort ?

- Counting Sort is a non-comparison based sorting algorithm.
- It counts the frequency of each element and uses that information to place elements in the correct position.
- It is very efficient when the range of input values is small.
- It has a time complexity of O(n + k) where k is the range of elements.
- It is a stable sorting algorithm.

How Counting Sort Works ?

1. Find the range (maximum value) in the array.
2. Create a count array of size (max + 1) and store the frequency of each element.
3. Modify the count array to store cumulative counts.
4. Place each element in its correct position using the count array.
5. Repeat for all elements.

Example (Step by Step)

Array:

4	2	2	8	3	3	1

Step 1: Find the range

Maximum element = 8 → create count array of size 9 (0 to 8)

Step 2: Count frequency

Index	0	1	2	3	4	5	6	7	8
Count	0	1	2	2	1	0	0	0	1

Step 3: Cumulative count (prefix sum)

Index	0	1	2	3	4	5	6	7	8
Count	0	1	3	5	6	6	6	6	7

Step 4: Build the output array

Output:

1	2	2	3	3	4	8

Algorithm (Pseudocode)

```
counting_sort(arr):
    k = max(arr)
    count = [0] * (k + 1)
    for num in arr:
        count[num] += 1
    for i = 1 to k:
        count[i] += count[i - 1]  # cumulative count
    output = [0] * len(arr)
    for num in reversed(arr):   # for stability
        output[count[num] - 1] = num
        count[num] -= 1
    return output
```

Complexity Analysis

Case	Time Complexity	Space Complexity	Explanation
Best Case	O(n + k)	O(k)	k = range of input values
Average Case	O(n + k)	O(k)	Same as best case
Worst Case	O(n + k)	O(k)	Same as best case

Advantages

- ✅ Very fast (O(n + k)).
- ✅ Stable sorting algorithm.
- ✅ Works well for small range of values.
- ✅ Simple to implement.

Disadvantages

- ❌ Not suitable for large range of values.
- ❌ Requires extra space O(k).
- ❌ Works only with non-negative integers.
- ❌ Not comparison based.

Key Takeaways

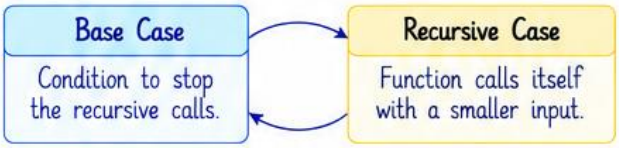
- ⭐ Heap Sort uses a Max Heap and has O(n log n) time complexity.
- ⭐ Counting Sort uses frequency counting and has O(n + k) time complexity.
- ⭐ Heap Sort is comparison based, Counting Sort is non-comparison based.
- ⭐ Counting Sort is very efficient when the range of input values is small.
- ⭐ Heap Sort is in-place, Counting Sort requires extra space.

10. Recursion + Backtracking

What is Recursion ?

- Recursion is a technique where a function calls itself to solve a smaller subproblem of the same problem.
- It breaks a big problem into smaller similar problems until a base case is reached.
- It is widely used in problems like tree traversal, dynamic programming, backtracking, etc.

Components of Recursion



How Recursion Works ?

1. Check if the base case is reached.
2. If not, solve a smaller subproblem by calling the function recursively.
3. Combine the result to get the final answer.

Example: Factorial using Recursion

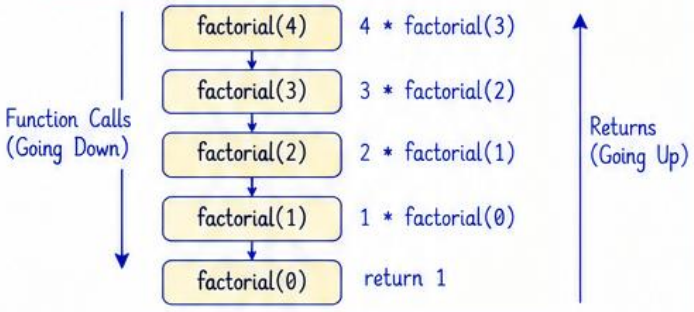
Python

```python
def factorial(n):
    if n == 0:        # base case
        return 1
    return n * factorial(n - 1)   # recursive case

# Example
print(factorial(5))  # Output: 120
```

@codedsout_05

Recursive Call Stack (n = 4)



Result: $4! = 4 \times 3 \times 2 \times 1 = 24$

Complexity Analysis

Case	Time Complexity	Space Complexity	Explanation
Best Case	O(n)	O(n)	n recursive calls
Average Case	O(n)	O(n)	n recursive calls
Worst Case	O(n)	O(n)	n recursive calls (stack space)

Advantages

- ✅ Simple and elegant code
- ✅ Easy to solve complex problems
- ✅ Widely used in trees, graphs, DP

Disadvantages

- ❌ Extra space due to call stack
- ❌ Can be slow for large inputs
- ❌ May cause stack overflow

What is Backtracking ?

- Backtracking is an algorithmic technique used to find all possible solutions to a problem by exploring different choices.
- It tries a solution, and if it does not lead to a valid answer, it backtracks and tries another option.
- It is commonly used in problems like N-Queens, Sudoku, Subsets, Permutations, etc.

Key Idea

- Build the solution step by step.
- If the current choice is not valid, undo the choice (backtrack) and try another option.
- Continue until all possibilities are explored.

General Template

```python
def solve(state):
    if is_solution(state):
        return
    for choice in possible_choices(state):
        if is_valid(state, choice):
            make_choice(state, choice)
            solve(state)           # recursive call
            undo_choice(state, choice)  # backtrack
```

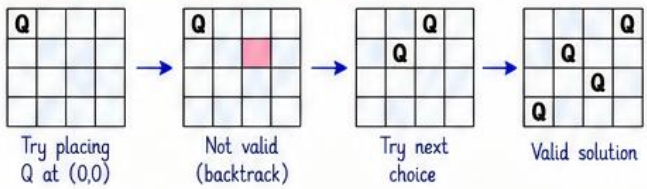
Example: N-Queens Problem (n = 4)

Python

```python
def solve_n_queens(row, board, n, res):
    if row == n:
        res.append([" ".join(r) for r in board])
        return
    for col in range(n):
        if is_safe(board, row, col, n):
            board[row][col] = 'Q'    # make choice
            solve_n_queens(row + 1, board, n, res)
            board[row][col] = '.'    # backtrack

# Example
n = 4
board = [['.' for _ in range(n)] for _ in range(n)]
res = []
solve_n_queens(0, board, n, res)
for config in res:
    print(config)
```

Visual Example (n = 4)



Complexity Analysis

Case	Time Complexity	Space Complexity	Explanation
Worst Case	$O(b^d)$	O(d)	b = choices, d = depth
Average Case	$O(b^d)$	O(d)	Explores all valid paths

Advantages

- ✅ Finds all possible solutions
- ✅ Useful for constraint-based problems
- ✅ Widely used in competitive programming

Disadvantages

- ❌ Can be slow (exponential time)
- ❌ Not suitable for very large inputs
- ❌ Requires careful pruning to be efficient

11. Two Pointer + Sliding Window

What is Two Pointer ?

- Two Pointer is a technique where we use two pointers to traverse the array, usually from start and end (or in the same direction).
- It is mainly used on sorted arrays or problems involving pairs or subarrays.
- It helps to reduce the time complexity from $O(n^2)$ to $O(n)$ in many problems.
- It is commonly used in problems like pair sum, remove duplicates, 3 sum, etc.

Types of Two Pointer

1. Opposite Direction

One pointer at start and one at end

2. Same Direction

Both pointers move in the same direction (i and j)

How Two Pointer Works ?

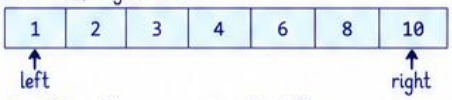
1. Initialize two pointers (left and right).
2. Check the condition and move pointers accordingly.
3. Continue until the pointers meet or the condition is satisfied.

Example 1: Pair Sum in Sorted Array

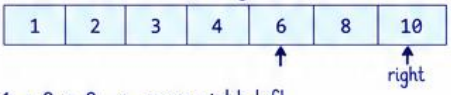
Find two numbers that add up to a target sum.

Array: | 1 | 2 | 3 | 4 | 6 | 8 | 10 | Target = 8

Step 1: left = 0, right = 6



Step 2: 1 + 10 > 8 → move right left



Step 3: 1 + 8 > 8 → move right left



Step 4: 1 + 6 = 8 → found pair (1, 6)



@codedsoul_05

Algorithm (Pseudocode)

```
left = 0, right = n - 1
while left < right:
    if arr[left] + arr[right] == target:
        return [arr[left], arr[right]]
    elif arr[left] + arr[right] < target:
        left += 1
    else:
        right -= 1
return []  # no pair found
```

Complexity Analysis

Case	Time Complexity	Space Complexity	Explanation
Best Case	O(n)	O(1)	Pair found early
Average Case	O(n)	O(1)	Linear traversal
Worst Case	O(n)	O(1)	Whole array traversed

Advantages

- ✅ Reduces time complexity
- ✅ Simple and efficient
- ✅ Works well on sorted arrays

Disadvantages

- ❌ Mostly applicable on sorted arrays
- ❌ Not suitable for all problems

What is Sliding Window ?

- Sliding Window is a technique used to find a subarray or substring that satisfies a given condition.
- It uses two pointers (left and right) to maintain a window of elements.
- The window is expanded to include more elements and shrunk when the condition is not satisfied.
- It is very useful for problems like maximum sum subarray, longest substring, subarray with at most k distinct elements, etc.

Types of Sliding Window

1. Fixed Size Window

Window size is constant (e.g. find maximum sum of subarray of size k)

2. Variable Size Window

Window size changes based on the condition (e.g. longest substring without repeating characters)

How Sliding Window Works ?

1. Expand the window by moving the right pointer.
2. Update the answer based on the current window.
3. Shrink the window from the left if the condition is not satisfied.
4. Repeat until the right pointer reaches the end.

Example 1: Maximum Sum Subarray of Size k (Fixed Size)

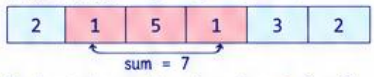
Array: | 2 | 1 | 5 | 1 | 3 | 2 | k = 3

Step 1: First window sum = 2 + 1 + 5 = 8

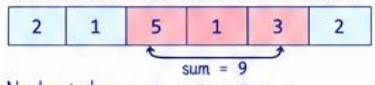


Window size = 3
Move right pointer and update sum

Step 2: Next window = 1 + 5 + 1 = 7



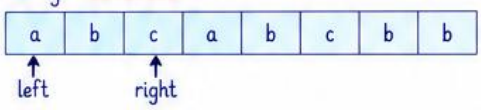
Step 3: Next window = 5 + 1 + 3 = 9 (max)



Step 4: Next window = 1 + 3 + 2 = 6

Example 2: Longest Substring Without Repeating Characters (Variable Size)

String: "abcabcbb"



Expand right and maintain a set.
Shrink left when duplicate is found.

Algorithm (Pseudocode) Fixed Size (Max Sum)

```
window_sum = 0
for i = 0 to n - 1:
    window_sum += arr[i]
    if i >= k: # remove leftmost element
        window_sum -= arr[i - k]
    ans = max(ans, window_sum)
return ans
```

Complexity Analysis

Case	Time Complexity	Space Complexity	Explanation
Fixed Size	O(n)	O(1)	Each element visited once
Variable Size	O(n)	O(k)	k = size of window / set

Advantages

- ✅ Efficient for subarray / substring problems
- ✅ Reduces time complexity
- ✅ Works for both fixed and variable size windows

Disadvantages

- ❌ Not applicable to all problems
- ❌ Can be tricky to implement
- ❌ Requires careful handling of conditions

12. Prefix Sum + Difference Array

What is Prefix Sum ?

- Prefix Sum is a technique where we precompute cumulative sums of an array.
- It helps to quickly find the sum of elements in any subarray [l, r] in O(1) time.
- Preprocessing takes O(n) time.
- It is widely used in problems related to subarray sum, range queries, counting, etc.

How Prefix Sum Works ?

1. Create a prefix array where prefix[i] stores the sum of elements from index 0 to i.
2. To find sum of subarray [l, r], use:
   sum = prefix[r] - prefix[l - 1] (if l > 0)
3. If l = 0, then sum = prefix[r].

Example: Build Prefix Sum Array

Array:	2	4	1	3	5
Prefix Array:	2	6	7	10	15
	2 (2)	2+4 (6)	6+1 (7)	7+3 (10)	10+5 (15)

Example: Range Sum Query

Find sum of elements from index l = 1 to r = 3

Array:	2	4	1	3	5
Prefix Array:	2	6	7	10	15

```
sum = prefix[r] - prefix[l - 1]
    = prefix[3] - prefix[0]
    = 10 - 2 = 8
```

@codedsoul_05

Code: Build Prefix Sum and Query

Python

```python
def build_prefix(arr):
    n = len(arr)
    prefix = [0] * n
    prefix[0] = arr[0]
    for i in range(1, n):
        prefix[i] = prefix[i - 1] + arr[i]
    return prefix

# Query sum in range [l, r]
def range_sum(prefix, l, r):
    if l == 0:
        return prefix[r]
    return prefix[r] - prefix[l - 1]
```

Complexity Analysis

Case	Time Complexity	Space Complexity	Explanation
Build Prefix	O(n)	O(n)	Single pass through array
Range Query	O(1)	O(1)	Using prefix difference

Advantages

- ✅ Very fast range sum queries
- ✅ Simple to implement
- ✅ Useful in many array problems
- ✅ Works well for static arrays

Disadvantages

- ❌ Only works for static arrays
- ❌ Not efficient for frequent updates
- ❌ Requires extra space O(n)
- ❌ Not directly usable for range updates

What is Difference Array ?

- Difference Array is a technique used for efficient range updates on an array.
- Instead of updating each element in the range, we update only two positions in a difference array.
- After all updates, we take the prefix sum to get the final array.
- It is useful when we have many range update queries and need the final array.

How Difference Array Works ?

1. Create a difference array diff of size n (initially all 0).
2. To add a value val in range [l, r]:
   diff[l] += val
   diff[r + 1] -= val (if r + 1 < n)
3. After all updates, take prefix sum of diff to get the final array.

Example: Range Updates

Initial Array (all zeros):

0	0	0	0	0

Operations:

1. Add 3 to range [1, 3]
2. Add 2 to range [0, 2]
3. Add 1 to range [2, 4]

Step 1: Apply updates on Difference Array

Diff Array (size 5, initialized to 0):

0	0	0	0	0

For range [l, r], we do:
diff[l] += val
diff[r + 1] -= val

Operation	Update on diff	Diff Array (after update)
Add 3 to [1, 3]	diff[1] += 3 diff[4] -= 3	0 3 0 0 -3
Add 2 to [0, 2]	diff[0] += 2 diff[3] -= 2	2 3 0 -2 -3
Add 1 to [2, 4]	diff[2] += 1 (no diff[5])	2 3 1 -2 -3

Step 2: Take prefix sum to get final array

Diff Array:	2	3	1	-2	-3
Final Array (after prefix sum):	2	5	6	4	1

Code: Range Updates using Difference Array

Python

```python
def apply_range_updates(n, queries):
    diff = [0] * n
    for l, r, val in queries:
        diff[l] += val
        if r + 1 < n:
            diff[r + 1] -= val
    # Build final array using prefix sum
    arr = [0] * n
    arr[0] = diff[0]
    for i in range(1, n):
        arr[i] = arr[i - 1] + diff[i]
    return arr
```

Complexity Analysis

Case	Time Complexity	Space Complexity	Explanation
Apply all updates	O(q)	O(n)	q = number of queries
Build final array	O(n)	O(1)	Prefix sum on diff
Total	O(n + q)	O(n)	Efficient for many updates

Advantages

- ✅ Efficient for multiple range updates
- ✅ Each update takes O(1) time
- ✅ Useful in competitive programming
- ✅ Simple and elegant technique

Disadvantages

- ❌ Only useful for range updates (not queries)
- ❌ Requires prefix sum step at the end
- ❌ Not suitable for dynamic updates online
- ❌ Less intuitive than direct update

13. Hashing + Hash-Based Algorithms

What is Hashing ?

- Hashing is a technique to store and retrieve data efficiently using a hash function.
- It converts a key into an index (hash value) to quickly access the value.
- It provides average O(1) time complexity for search, insert and delete operations.
- It is widely used in problems like duplicates, frequency count, pair sum, etc.

How Hashing Works ?

1. A hash function converts a key into a fixed size hash value (index).
2. The value is stored at that index in a hash table.
3. For the same key, the hash function always returns the same index.
4. If two different keys map to the same index, it is called a collision.

Hash Table Example

Key	→	Hash Function	→	Index
key = 15	→	15 % 7	→	1
key = 22	→	22 % 7	→	1
key = 8	→	8 % 7	→	1
key = 3	→	3 % 7	→	3

Hash Table (size = 7)

0	-
1	15 → 22 → 8
2	-
3	3
4	-
5	-
6	-

Multiple values at same index are stored using chaining (linked list).

@codedsoul_05

Types of Hashing

1. Hashing with Chaining

- Each index stores a list of values (linked list or dynamic array).
- Handles collisions.
- Simple and widely used.

2. Hashing with Open Addressing

- If a collision occurs, find another empty slot (linear probing, quadratic probing, etc).
- All values are stored in the array itself.
- Used when space is limited.

Common Hash-Based Problems

1. Check if an element exists in array.
2. Find duplicates in an array.
3. Find frequency of elements.
3. Find pair with given sum.
5. Find longest subarray with sum = k (using prefix sum + hash).
6. Find first non-repeating character.
7. Group anagrams.
8. Count subarrays with given sum.

Basic HashMap Operations (Python)

Python

```python
# Create a hash map
mp = {}
# Insert key-value
mp["apple"] = 5
# Access value
print(mp["apple"])  # Output: 5
# Update value
mp["apple"] = 10
# Check if key exists
print("apple" in mp)  # Output: True
# Iterate
for key, value in mp.items():
    print(key, value)
# Delete key
del mp["apple"]
```

Example 1: Find Duplicates in an Array

Array: `4, 2, 7, 8, 2, 3, 1, 4]`

Output: `[2, 4]`

Python

```python
def find_duplicates(arr):
    mp = {}
    res = []
    for num in arr:
        if num in mp:
            res.append(num)
        else:
            mp[num] = 1
    return res

arr = [4, 2, 7, 8, 2, 3, 1, 4]
print(find_duplicates(arr))  # Output: [2, 4]
```

Example 2: Find Pair with Given Sum

Array: `8, 4, 7, 2, 5, 3]` Target = 9

Python

```python
def find_pair(arr, target):
    mp = {}
    for num in arr:
        complement = target - num
        if complement in mp:
            return [complement, num]
        mp[num] = 1
    return []

arr = [8, 4, 7, 2, 5, 3]
print(find_puplicarr, 9)) # Output: [4, 5]
```

Example 3: Longest Subarray with Sum = k

Array: `1, 2, 3, 1, 1, 1]` k = 3

Python

```python
def longest_subarray_sum_k(arr, k):
    mp = {0: -1}
    curr_sum = 0
    ans = 0
    for i, num in enumerate(arr):
        curr_sum += num
        if curr_sum - k in mp:
            ans = max(ans, i - mp[curr_sum - k])
        if curr_sum not in mp:
            mp[curr_sum] = i
    return ans

arr = [1, 2, 3, 1, 1, 1]
print(longest_subarray_sum_k(arr, 3))  # Output: 4
```

Complexity Analysis

Operation	Average Case	Worst Case	Explanation
Insert	O(1)	O(n)	Collision leads to chain traversal
Search	O(1)	O(n)	May need to traverse chain
Delete	O(1)	O(n)	May need to traverse chain

Advantages

- ✅ Very fast average time complexity
- ✅ Simple to implement
- ✅ Used in many real world problems
- ✅ Supports efficient search, insert, delete

Disadvantages

- ❌ Worst case can be O(n)
- ❌ Requires extra space
- ❌ Performance depends on hash function
- ❌ Not suitable when order matters

Key Takeaways

- ⭐ Hashing maps a key to an index using a hash function.
- ⭐ It provides average O(1) time complexity.
- ⭐ Hash-based algorithms are useful for duplicates, frequency, pair sum, subarray sum and many more problems.
- ⭐ Handling collisions is important for good performance.

14. Greedy Algorithms

What is Greedy Algorithm ?

- Greedy algorithm is a problem solving technique where we make the locally optimal choice at each step with the hope of finding the global optimal solution.
- It does not consider all possible solutions like brute force or dynamic programming.
- It is simple, efficient and works well for many optimization problems.
- It may not always give the optimal solution for every problem.

Key Idea

1. Make the best possible choice at the current step.
2. Do not worry about future choices.
3. Hope that local optimal choice leads to global optimal solution.
4. Continue until the problem is solved.

When to Use Greedy ?

- When the problem has optimal substructure.
- When the greedy choice property holds (local optimal choice leads to global optimal solution).
- Commonly used in interval scheduling, activity selection, Huffman coding, MST (Kruskal, Prim), etc.

Greedy vs Other Approaches

Approach	Idea	Time Complexity	Optimal ?
Brute Force	Try all possibilities	Exponential	Yes
Dynamic Programming	Solve subproblems	Polynomial	Yes
Greedy Algorithm	Make best local choice	Usually O(n log n)	Not always

Example 2: Fractional Knapsack

Given weights and values of n items and a knapsack with capacity W. Find the maximum value we can obtain (items can be taken in fraction).

Input:

Weight:	10	20	30
Value:	60	100	120

Capacity W = 50

Step 1: Calculate value per unit weight (value / weight)

Item	Weight	Value	Value / Weight
1	10	60	6.0
2	20	100	5.0
3	30	120	4.0

Step 2: Sort by value / weight (descending) and pick items

Take item 1 (10, 60) → remaining capacity = 40

Take item 2 (20, 100) → remaining capacity = 20

Take 20/30 of item 3 → value = 120 × (20/30) = 80

Total Value = 60 + 100 + 80 = 240

Code (Python)

Python

```python
def fractional_knapsack(weights, values, W):
    items = sorted(zip(values, weights), key=lambda x: x[0]/x[1],
                   reverse=True)
    total_value = 0.0
    for v, w in items:
        if W == 0:
            break
        take = min(w, W)
        total_value += (v / w) * take
        W -= take
    return total_value
```

Example 1: Activity Selection Problem

Given n activities with start and end times, select the maximum number of non overlapping activities.

Input:

Start:	1	3	0	5	8	5
End:	2	4	6	7	9	9

Approach:

1. Sort activities by their end time.
2. Pick the first activity.
3. For each next activity, if its start time is greater than or equal to the end time of the last selected activity, pick it.
4. Repeat until all activities are processed.

Selected Activities:

(1, 2)	(3, 4)	(5, 7)	(8, 9)

Output: 4 (maximum number of activities)

Code (Python)

Python

```python
def activity_selection(start, end):
    activities = sorted(zip(end, start), key=lambda x: x[0])
    count = 0
    last_end = -1
    for e, s in activities:
        if s >= last_end:
            count += 1
            last_end = e
    return count
```

Example 3: Minimum Coins (Canonical Problem)

Given a set of coin denominations and a target amount. Find the minimum number of coins to make the amount (works for canonical coin systems like 1, 5, 10, 25, ...).

Input:

Coins:	1	5	10	25

Amount = 37

Approach:

1. Sort coins in descending order.
2. Always pick the largest coin less than or equal to the remaining amount.
3. Subtract the coin value and continue.
4. Repeat until the amount becomes 0.

Coin Selection:

Take 1 coin of 25 → remaining = 12

Take 1 coin of 10 → remaining = 2

Take 2 coins of 1 → remaining = 0

Total Coins = 4

Complexity Analysis

Problem	Time Complexity	Space Complexity	Optimal ?
Activity Selection	O(n log n)	O(1)	Yes
Fractional Knapsack	O(n log n)	O(1)	Yes
Minimum Coins (canonical)	O(n)	O(1)	Yes
General Greedy	Varies	Varies	Not always

Advantages

- ✓ Simple to implement
- ✓ Efficient (often O(n) or O(n log n))
- ✓ Works well for many optimization problems
- ✓ Used in real world applications

Disadvantages

- ✗ Does not always give optimal solution
- ✗ Works only when greedy choice property holds
- ✗ May fail for some problems (e.g. 0/1 Knapsack)
- ✗ Need to prove correctness for each problem

Key Takeaways

- ★ Greedy algorithm makes the locally optimal choice at each step.
- ★ It works when the problem has optimal substructure and greedy choice property.
- ★ It is simple and efficient but does not always guarantee the optimal solution.
- ★ Common applications include interval scheduling, knapsack (fractional), MST, Huffman coding, etc.

15. Dynamic Programming Fundamentals

Algorithms Series Rage 14

What is Dynamic Programming ?

- Dynamic Programming (DP) is an optimization technique used to solve problems that have overlapping subproblems and optimal substructure.
- It breaks a big problem into smaller subproblems, solves each subproblem once and stores the result (memoization or tabulation) to avoid redundant work.
- It is mainly used on problems where brute force leads to exponential time complexity.
- It is widely used in competitive programming, interview problems and real world optimization.

Key Characteristics

1. **Overlapping Subproblems** : Same subproblems are solved again and again.
2. **Optimal Substructure** : Optimal solution can be built from optimal solutions of smaller subproblems.

When to Use DP ?

1. When the problem can be divided into smaller subproblems.
2. When subproblems overlap.
3. When the problem has optimal substructure.
4. When brute force gives exponential time complexity.

Steps to Solve a DP Problem

1. Define the state (what does dp[i] represent ?)
2. Write the recurrence relation (transition).
3. Decide on base case(s).
4. Implement using memoization (top down) or tabulation (bottom up).
5. Return the answer from the final state.

Top Down vs Bottom Up

Top Down (Memoization)

- Use recursion + cache (dp array)
- Solve only required states
- More intuitive to write
- May have recursion stack overhead

Bottom Up (Tabulation)

- Solve states iteratively
- Fill dp table from base case
- No recursion stack overhead
- Often more space efficient

General Template (Top Down)

Python

```python
def solve(state):
    if state in dp:
        return dp[state]      # already computed
    if base_case(state):
        return base_value     # base case
    # try all possible choices and store the best answer
    ans = recurrence_using_subproblems(state)
    dp[state] = ans
    return ans
```

General Template (Bottom Up)

Python

```python
# initialize dp table
for base states:
    dp[state] = base_value

# fill states in increasing order
for state in all_states:
    for each choice in possible_choices:
        dp[state] = combine(dp[previous_states])

return dp[final_state]
```

Example 1: Fibonacci Sequence

Find the nth Fibonacci number.
F(n) = F(n-1) + F(n-2), F(0) = 0, F(1) = 1

Top Down (Memoization)

Python

```python
def fib(n, dp):
    if n <= 1:
        return n
    if n in dp:
        return dp[n]
    dp[n] = fib(n - 1, dp) +
            fib(n - 2, dp)
    return dp[n]
```

Bottom Up (Tabulation)

Python

```python
def fib(n):
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
      dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]
```

Time Complexity : O(n)
Space Complexity : O(n) (can be O(1) with optimization)

Example 2: 0/1 Knapsack

Given n items with weights wt[i] and values val[i] and a capacity W.
Find the maximum value we can obtain (each item can be taken at most once).

Top Down (Memoization)

Python

```python
def knapsack(i, W, wt, val, dp):
    if n == 1 or W == 0:
        return 0
    if dp[i][W] != -1:
        return dp[i][W]
    if wt[i - 1] <= W:
        take = val[i - 1] +
               knapsack(i - 1, W - wt[i - 1],
                wt, val, dp)
        not_take = knapsack(i - 1, W, wt, val, dp)
        dp[i][W] = max(take, not_take)
    else:
        dp[i][W] = knapsack(i - 1, W, wt, val, dp)
    return dp[i][W]
```

@codedsoul_05

Bottom Up (Tabulation)

Python

```python
def knapsack(wt, val, W):
    n = len(wt)
    dp = [[0] * (W + 1)
          for _ in range(n + 1)]
    for i in range(1, n + 1):
      for w in range(0, W + 1):
        if wt[i - 1] <= w:
          take = val[i - 1] +
            dp[i - 1][w - wt[i - 1]]
          not_take = dp[i - 1][w]
          dp[i][w] = max(take, not_take)
        else:
          dp[i][w] = dp[i - 1][w]
    return dp[n][W]
```

Time Complexity : O(nW)
Space Complexity : O(nW) (can be optimized to O(W))

Example 3: Longest Common Subsequence (LCS)

Given two strings, find the length of the longest subsequence that appears in both strings.

Top Down (Memoization)

Python

```python
def lcs(i, j, s1, s2, dp):
    if i == 0 or j == 0:
        return 0
    if dp[i][j] != -1:
        return dp[i][j]
    if s1[i - 1] == s2[j - 1]:
      dp[i][j] = 1 + lcs(i - 1, j - 1, s1, s2, dp)
    else:
        dp[i][j] = max(lcs(i - 1, j, s1, s2, dp),
                       lcs(i, j - 1, s1, s2, dp))
    return dp[i][j]
```

Bottom Up (Tabulation)

Python

```python
def lcs(s1, s2):
    n, m = len(s1), len(s2)
    dp = [[0] * (m + 1)
          for _ in range(n + 1)]
    for i in range(1, n + 1):
      for j in range(1, m + 1):
        if s1[i - 1] == s2[j - 1]:
          dp[i][j] = 1 + dp[i - 1][j - 1]
        else:
          dp[i][j] = max(dp[i - 1][j,
                         dp[i][j - 1])
    return dp[n][m]
```

Time Complexity : O(nm)
Space Complexity : O(nm) (can be optimized to O(min(n, m)))

Complexity Comparison

Approach	Time Complexity	Space Complexity	Pros	Cons
Naive Recursion	Exponential	O(n)	Simple to write	Very slow
Top Down (Memoization)	Polynomial	O(n) or O(nm)	Solves only needed states	Recursion overhead
Bottom Up (Tabulation)	Polynomial	O(n) or O(nm)	No recursion overhead	May compute extra states

Advantages

- ✅ Reduces time complexity from exponential to polynomial
- ✅ Solves overlapping subproblems efficiently
- ✅ Widely applicable in optimization problems
- ✅ Works well for sequential decision making

Disadvantages

- ❌ Can be harder to identify DP state and recurrence
- ❌ Sometimes uses more space
- ❌ May compute unnecesary states (bottom up)
- ❌ Not always applicable to all problems

Key Takeaways

- ⭐ Use DP when the problem has overlapping subproblems and optimal substructure.
- ⭐ Identify the state, write the recurrence and handle base cases.
- ⭐ Choose memoization for simplicity or tabulation for better space control.
- ⭐ Practice classic DP problems like Fibonacci, Knapsack, LCS, Coin Change, etc.

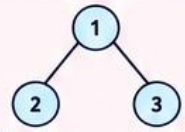
16. Graph Algorithms + BFS + DFS

What is a Graph ?

- A graph is a non linear data structure consisting of nodes (vertices) and edges.
- It can be directed or undirected.
- Graphs are used to represent real world problems like social networks, maps, dependencies, etc.
- Commonly used in problems like search, shortest path, connected components, cycle detection, etc.

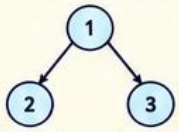
Types of Graphs

1. Undirected Graph



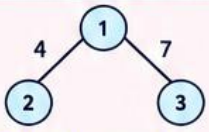
Edges have no direction

2. Directed Graph



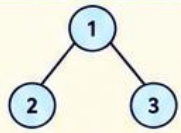
Edges have direction

3. Weighted Graph



Edges have weights

4. Unweighted Graph



All edges have equal weight

Graph Representation

1. Adjacency List (Preferred)

```
graph = {
  1: [2, 3],
  2: [1, 4],
  3: [1],
  4: [2]
}
```

- Space: O(V + E)
- Good for sparse graphs
- Easy to iterate neighbors

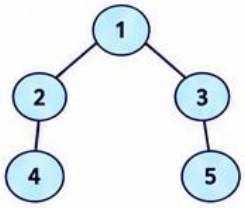
2. Adjacency Matrix

	0	1	2	3
0	0	1	1	0
1	1	0	1	0
2	1	1	0	0
3	0	0	0	0

- Space: O(V^2)
- Good for dense graphs

Breadth First Search (BFS)

- BFS explores the graph level by level using a queue.
- It finds the shortest path (in unweighted graphs).
- It is widely used in shortest path, level order traversal, connected components, etc.



BFS Traversal (start = 1)

Order: 1 → 2 → 3 → 4 → 5

Level by level:

- Level 0: 1
- Level 1: 2, 3
- Level 2: 4, 5

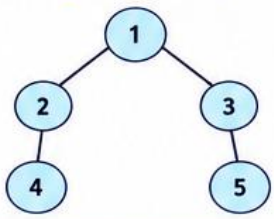
BFS Code (Python)

Python

```python
from collections import deque
def bfs(graph, start):
    visited = set()
    q = deque([start])
    visited.add(start)
    order = []
    while q:
        node = q.popleft()
        order.append(node)
        for neigh in graph[node]:
            if neigh not in visited:
                visited.add(neigh)
                q.append(neigh)
    return order
```

Depth First Search (DFS)

- DFS explores as far as possible along a branch before backtracking.
- It can be implemented using recursion or a stack.
- It is used in connected components, cycle detection, topological sort, strongly connected components, etc.



DFS Traversal (start = 1)

One possible order:
1 → 2 → 4 → 3 → 5
(DFS order can vary based on adjacency list)

DFS Code (Python - Recursive)

Python

```python
def dfs(graph, node, visited, order):
    visited.add(node)
    order.append(node)
    for neigh in graph[node]:
        if neigh not in visited:
            dfs(graph, neigh, visited,
            order)

# Usage
visited = set()
order = []
dfs(graph, 1, visited, order)
print(order)
```

DFS Code (Python - Iterative)

Python

```python
def dfs_iterative(graph, start):
    visited = set()
    stack = [start]
    order = []
    while stack:
        node = stack.pop()
        if node not in visited:
            visited.add(node)
            order.append(node)
        # add neighbors in reverse to
        # maintain same order as recursive
        for neiigh in reversed(graph[node]):
            if neigh not in visited:
                stack.append(neigh)
    return order
```

@codedsoul_05

Common Graph Problems

1. Find if a path exists between two nodes (BFS or DFS)
2. Find shortest path in unweighted graph (BFS)
3. Count number of connected components (DFS)
4. Detect cycle in a graph (DFS)
5. Topological sort (DFS)
6. Find strongly connected components (Kosaraju, Tarjan)
7. Minimum spanning tree (Prim, Kruskal)
8. Single source shortest path in weighted graph (Dijkstra)

Complexity Analysis

Algorithm	Time Complexity	Space Complexity	Explanation
BFS	O(V + E)	O(V)	Visit each node and edge once
DFS	O(V + E)	O(V)	Visit each node and edge once
Adjacency List	O(V + E)	O(V + E)	Good for sparse graphs
Adjacency Matrix	O(V^2)	O(V^2)	Good for dense graphs

Advantages

- ✅ Useful for many real world problems
- ✅ BFS finds shortest path in unweighted graphs
- ✅ DFS helps in exploration and cycle detection
- ✅ Graphs can model complex relationships

Disadvantages

- ❌ Can be complex to implement
- ❌ May require extra space
- ❌ Not always optimal for weighted graphs (need Dijkstra or other algorithms)
- ❌ DFS recursion may cause stack overflow for large graphs

Key Takeaways

- ⭐ Use BFS when you need shortest path or level wise traversal.
- ⭐ Use DFS when you need to explore all possibilities or detect cycles.
- ⭐ Choose adjacency list for sparse graphs and adjacency matrix for dense graphs.
- ⭐ Graphs are a powerful tool and are used in many real world and interview problems.

17. Shortest Path Algorithms + Dijkstra + Bellman Ford

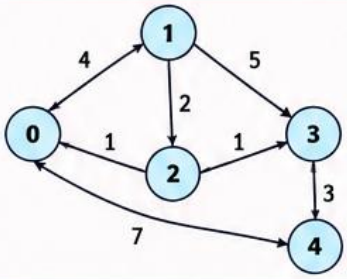
What is Shortest Path ?

- Shortest path problem is to find the minimum cost path between two nodes in a graph.
- Each edge has a weight (cost), and we want the path with the smallest total weight.
- Used in real world problems like navigation, network routing, logistics, etc.

Types of Shortest Path Problems

1. Single Source Shortest Path : From one source to all nodes.
2. Single Pair Shortest Path : From one source to one destination.
3. All Pairs Shortest Path : Shortest path between every pair of nodes (Floyd Warshall).

Graph Example



Adjacency List

0 : [(1, 4), (2, 1), (4, 7)]
1 : [(2, 2), (3, 5)]
2 : [(3, 1)]
3 : [(4, 3)]
4 : []

(node, weight)

@codedsoul_05

Dijkstra's Algorithm

- Finds shortest path from a single source to all other nodes in a graph with non negative edge weights.
- Uses a min priority queue (min heap) to always pick the node with minimum distance.
- Time Complexity : O((V + E) log V)
- Does not work with negative edge weights.

How it Works ?

1. Initialize distances with infinity, except source = 0.
2. Use a min heap to pick the node with smallest distance.
3. Update distances of its neighbors if a shorter path is found.
4. Repeat until all nodes are visited.

Dijkstra's Code (Python)

Python

```python
import heapq
def dijkstra(n, graph, src):
    dist = [float('inf')] * n
    dist[src] = 0
    pq = [(0, src)]  # (distance, node)
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if dist[v] > d + w:
                dist[v] = d + w
                heapq.heappush(pq, (dist[v], v))
    return dist
```

Comparison

Algorithm	Handles Negative Weights	Detects Negative Cycle	Time Complexity	Use Case
Dijkstra	No	No	O((V + E) log V)	Shortest path (non negative)
Bellman Ford	Yes	Yes	O(V × E)	Shortest path (negative weights)
Floyd Warshall	Yes	Yes	O(V^3)	All pairs shortest path

Dijkstra Example (Source = 0)

Node	Initial Dist	After Step 1	After Step 2	After Step 3	After Step 4	Final Dist
0	0	0	0	0	0	0
1	∞	4	3	3	3	3
2	∞	1	1	1	1	1
3	∞	∞	6	2	2	2
4	∞	7	7	5	5	5

Shortest distances from node 0: [0, 3, 1, 2, 5]

Bellman Ford Algorithm

- Finds shortest path from a single source to all other nodes, can handle negative edge weights.
- Works by relaxing all edges (V - 1) times.
- Can also detect negative weight cycles.
- Time Complexity : O(V × E)
- Slower than Dijkstra but more general.

How it Works ?

1. Initialize distances with infinity, except source = 0.
2. Relax all edges (V - 1) times.
3. Check for negative weight cycles by relaxing edges once more.
4. If any distance can still be updated, a negative cycle exists.

Bellman Ford Code (Python)

Python

```python
def bellman_ford(n, edges, src):
    dist = [float('inf')] * n
    dist[src] = 0
    # Relax all edges V - 1 times
    for _ in range(n - 1):
        updated = False
        for u, v, w in edges:
            if dist[u] !=float('inf') and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                updated = True
        if not updated:
            break
    # Check for negative weight cycle
    for u, v, w in edges:
        if dist[u] != float('inf') and dist[u] + w < dist[v]:
            return None  # Negative cycle exists
    return dist
```

Example (Bellman Ford with Negative Weights)

Edges (u, v, w):

0 → 1 (6)
0 → 2 (7)
1 → 2 (8)
1 → 3 (5)
1 → 4 (-4)
2 → 3 (-3)
3 → 1 (-2)
4 → 3 (7)

Iteration	d[0]	d[1]	d[2]	d[3]	d[4]
Initial	0	∞	∞	∞	∞
1	0	6	7	4	2
2	0	2	7	4	2
3	0	2	7	4	2
4	0	2	7	4	2

Shortest distances from node 0: [0, 2, 7, 4, 2]

Advantages

- ✅ Dijkstra is fast for non negative weights.
- ✅ Bellman Ford works with negative weights.
- ✅ Bellman Ford can detect negative cycles.
- ✅ Both are fundamental graph algorithms.

Disadvantages

- ❌ Dijkstra does not work with negative weights.
- ❌ Bellman Ford is slower (O(V × E)).
- ❌ Bellman Ford is not suitable for very large graphs.
- ❌ Both require graph representation (adjacency list or edge list).

Key Takeaways

- ⭐ Use Dijkstra when all edge weights are non negative.
- ⭐ Use Bellman Ford when graph has negative weights or to detect negative cycles.
- ⭐ Shortest path algorithms are widely used in real world applications like GPS, network routing, and logistics.

18. Minimum Spanning Tree + Kruskal + Prim

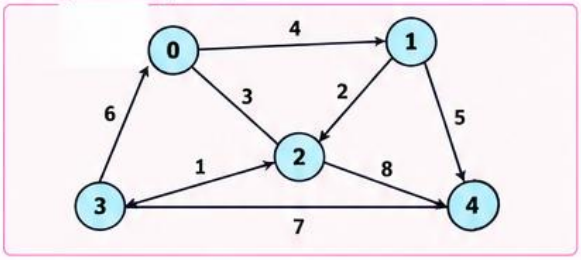
What is Minimum Spanning Tree (MST) ?

- A Minimum Spanning Tree is a subset of edges that connects all the vertices of a connected, undirected, weighted graph.
- It includes exactly (V - 1) edges, has no cycles, and gives the minimum possible total weight.
- It is widely used in network design, road construction, communication networks, etc.

Key Properties

1. Connects all vertices.
2. Contains (V - 1) edges.
3. Has no cycles.
4. Gives the minimum total weight.
5. For a connected graph, MST is unique if all edge weights are distinct (otherwise multiple MSTs may exist).

Example Graph



MST using Kruskal's Algorithm

- Sort all edges by weight in increasing order.
- Use Disjoint Set Union (Union-Find) to add edges without creating cycles.
- Keep adding edges until we have (V - 1) edges.
- Time Complexity : O(E log E)
- Space Complexity : O(V)

Steps (Kruskal)

1. Sort all edges by weight.
2. Initialize Disjoint Set (each node is a separate set).
3. Pick the smallest edge. If it connects two different components, add it to MST.
4. Repeat until (V - 1) edges are added.

Kruskal's Code (Python)

Python

```python
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n
    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]
    def union(self, x, y):
        px, py = self.find(x), self.find(y)
        if px == py:
            return False
        if self.rank[px] < self.rank[py]:
            self.parent[px] = py
        elif self.rank[px] > self.rank[py]:
            self.parent[py] = px
        else:
            self.parent[py] = px
            self.rank[px] += 1
        return True
def kruskal(n, edges):
    edges.sort(key=lambda x: x[2])   # sort by weight
    dsu = DSU(n)
    mst = []
    total_weight = 0
    for u, v, w in edges:
        if dsu.union(u, v):
            mst.append((u, v, w))
            total_weight += w
            if len(mst) == n - 1:
                break
    return mst, total_weight
```

Kruskal's Algorithm on Example

Edges (sorted by weight)

```
(3, 2, 1)
(1, 2, 2)
(0, 2, 3)
(0, 1, 4)
(1, 4, 5)
(0, 3, 6)
(3, 4, 7)
(2, 4, 8)
```

Step	Edge (u, v)	Weight	Action	MST Edges	Total
1	(3, 2)	1	Add	{(3,2)}	1
2	(1, 2)	2	Add	{(3,2), (1,2)}	3
3	(0, 2)	3	Add	{(3,2), (1,2), (0,2)}	6
4	(0, 1)	4	Skip	-	-
5	(1, 4)	5	Add	{(3,2), (1,2), (0,2), (1,4)}	11

MST Edges : (3, 2, 1) (1, 2, 2) (0, 2, 3) (1, 4, 5)
Total Weight : 11

MST using Prim's Algorithm

- Start from any node and grow the MST by always picking the minimum weight edge that connects a new node.
- Uses a min heap (priority queue) to pick the next minimum edge.
- Time Complexity : O((V + E) log V)
- Space Complexity : O(V)

Steps (Prim)

1. Start from any node (e.g., 0).
2. Use a min heap to store edges (weight, node, parent).
3. Pick the minimum weight edge that connects a new (unvisited) node.
4. Add it to MST and push its neighbors.
5. Repeat until all nodes are visited.

Prim's Code (Python)

Python

@codeosoul_05

```python
import heapq
def prim(n, adj, start=0):
    visited = [False] * n
    min_heap = [(0, start, -1)]  # (weight, node, parent)
    mst = []
    total_weight = 0
    while min_heap:
        weight, node, parent = heapq.heappop(min_heap)
        if visited[node]:
            continue
        visited[node] = True
        if parent != -1:
            mst.append((parent, node, weight))
            total_weight += weight
        for nei, w in adj[node]:
            if not visited[nei]:
                heapq.heappush(min_heap, (w, nei, node))
    return mst, total_weight
```

Prim's Algorithm on Example (Start = 0)

Step	Picked Edge (u, v)	Weight	New Node	MST Edges	Total
1	(0, 2)	3	2	{(0,2)}	3
2	(2, 3)	1	3	{(0,2), (2,3)}	4
3	(2, 1)	2	1	{(0,2), (2,3), (2,1)}	6
4	(1, 4)	5	4	{(0,2), (2,3), (2,1), (1,4)}	11

Total Weight : 11 (same as Kruskal)

Comparison

Feature	Kruskal	Prim
Approach	Sort all edges, use DSU	Grow MST from a starting node
Data Structure	Disjoint Set Union	Min Heap (Priority Queue)
Time Complexity	O(E log E)	O((V + E) log V)
Space Complexity	O(V)	O(V)
Better For	Sparse graphs	Dense graphs
Use Case	When edges are given as list	When graph is given as adjacency list

Advantages

- ✅ Efficient way to connect all nodes with minimum cost
- ✅ Used in real world applications (networks, roads, etc.)
- ✅ Kruskal is simple and works well for sparse graphs
- ✅ Prim is effficient for dense graphs

Disadvantages

- ❌ Both require connected graph
- ❌ Kruskal can be slower for dense graphs
- ❌ Prim requires adjacency list representation
- ❌ Does not work for directed graphs

Key Takeaways

- ⭐ MST connects all nodes with minimum total weight.
- ⭐ Kruskal uses edge sorting and DSU, Prim uses a min heap.
- ⭐ Both have different use cases but give the same MST cost.
- ⭐ MST is important in network design, road construction, and many real world problems.

19. Topological Sort + Advanced Graph Algorithms

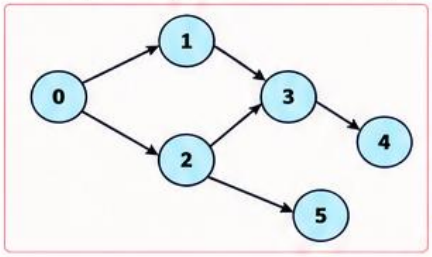
What is Topological Sort ?

- Topological Sort is a linear ordering of vertices in a Directed Acyclic Graph (DAG) such that for every edge u → v, u appears before v in the ordering.
- It is only possible for directed graphs with no cycles.
- Commonly used in task scheduling, dependency resolution, course prerequisite problems, etc.

Properties

1. Works only for Directed Acyclic Graph (DAG).
2. There can be multiple valid topological orders.
3. If there is a cycle, topological sort is not possible.
4. Time Complexity : O(V + E)
5. Space Complexity : O(V)

Example Graph (Course Prerequisites)



Adjacency List

0 : [1, 2]
1 : [3]
2 : [3, 5]
3 : [4]
4 : []
5 : []

One Topological Order : 0, 1, 2, 3, 5, 4 (Another valid order : 0, 2, 1, 5, 3, 4)

Kahn's Algorithm (BFS based)

- Maintain in-degree of all nodes.
- Add all nodes with in-degree 0 to a queue.
- Remove node from queue, reduce in-degree of its neighbors.
- Repeat until all nodes are processed.

Kahn's Algorithm Code (Python)

Python

```python
from collections import deque
def topo_sort_kahn(n, adj):
    indegree = [0] * n
    for u in range(n):
        for v in adj[u]:
            indegree[v] += 1
    q = deque([i for i in range(n) if indegree[i] == 0])
    topo = []
    while q:
        node = q.popleft()
        topo.append(node)
        for nei in adj[node]:
            indegree[nei] -= 1
            if indegree[nei] == 0:
                q.append(nei)
    if len(topo) != n:
        return []   # cycle exists
    return topo
```

DFS Based Topological Sort

- Perform DFS and push each node to a stack after visiting all its neighbors.
- Reverse the stack to get topological order.

DFS Topological Sort Code (Python)

Python

```python
def topo_sort_dfs(n, adj):
    visited = [False] * n
    stack = []

    def dfs(u):
        visited[u] = True
        for v in adj[u]:
            if not visited[v]:
                dfs(v)
        stack.append(u)

    for i in range(n):
        if not visited[i]:
            dfs(i)

    stack.reverse()
    return stack
```

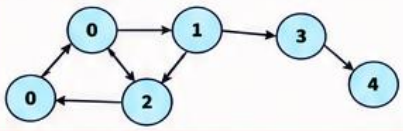
Advanced Graph Algorithms

- These algorithms solve more complex graph problems beyond basic traversal and shortest path.
- They are widely used in competitive programming, system design and real world applications.

1 Strongly Connected Components (SCC) – Kosaraju's Algorithm

- Finds strongly connected components in a directed graph.
- A strongly connected component (SCC) is a group of nodes where every node is reachable from every other node.
- Time Complexity : O(V + E)

Example Graph



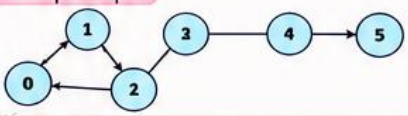
SCCs in the graph:

Component 1 : {0, 1, 2}
Component 2 : {3}
Component 3 : {4}

2 Bridges and Articulation Points (Tarjan's Algorithm)

- Bridge : An edge whose removal increases the number of connected components.
- Articulation Point : A node whose removal increases the number of connected components.
- Time Complexity : O(V + E)

Example Graph



Result:

Bridges : (1, 3), (3, 4), (4, 5)
Articulation Points : 1, 3, 4

@codedsoul_05

3 Disjoint Set Union (Union Find)

- Maintains disjoint sets and supports union and find operations.
- Used in Kruskal's algorithm, dynamic connectivity, etc.
- Time Complexity : O(α(V)) ~ O(1) (with path compression)

Disjoint Set Union Code (Python)

Python

```python
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])  # path compression
        return self.parent[x]

    def union(self, x, y):
        px, py = self.find(x), self.find(y)
        if px == py:
            return False
        if self.rank[px] < self.rank[py]:
            self.parent[px] = py
        elif self.rank[px] > self.rank[py]:
            self.parent[py] = px
        else:
            self.parent[py] = px
            self.rank[px] += 1
        return True
```

4 Other Important Advanced Graph Algorithms

Algorithm	Problem Solved	Time Complexity	Use Case
Topological Sort	Ordering in DAG	O(V + E)	Task scheduling, course prerequisites
SCC (Kosaraju/Tarjan)	Find strongly connected components	O(V + E)	Dependency analysis, compilers
Bridges & Articulation Points	Find critical edges/nodes	O(V + E)	Network resilience
Disjoint Set Union (DSU)	Maintain disjoint sets	O(α(V))	Kruskal, dynamic connectivity
Eulerian Path/Circuit	Path using every edge once	O(V + E)	Route planning
Hamiltonian Path/Cycle	Path visiting every node once	NP-Complete	Hard problem (theoretical)
Bipartite Check (2-Coloring)	Check if graph is bipartite	O(V + E)	Scheduling, matching
Graph Coloring	Assign colors to nodes	NP-Complete	Register allocation, timetabling
Minimum Cost Flow	Flow with minimum cost	Varies	Network design, logistics

Key Takeaways

- Topological sort works only for DAG.
- SCC, bridges, articulation points help analyze graph structure.
- DSU is a powerful tool for connectivity and MST (Kruskal).
- Advanced graph algorithms are useful in real world applications.

Applications

- Task scheduling and dependencies
- Network design and resilience
- Compiler design and dependency analysis
- Social network analysis
- Road and communication networks

20. Algorithm Cheat Sheet + 100 Interview Questions

Common Time Complexities

Notation	Time	Example
O(1)	Constant	Access array element
O(log n)	Logarithmic	Binary search
O(n)	Linear	Traverse array
O(n log n)	Linearithmic	Merge sort, Heap sort
O(n^2)	Quadratic	Nested loops, Bubble sort
O(n^3)	Cubic	Floyd Warshall
O(2^n)	Exponential	Subset generation, TSP (brute)
O(n!)	Factorial	Permutations

Common Space Complexities

Notation	Space	Example
O(1)	Constant	In-place variables
O(n)	Linear	Array, HashMap, Recursion stack
O(n^2)	Quadratic	Adjacency matrix
O(log n)	Logarithmic	Binary search (iterative)

Data Structure Operations

Data Structure	Access	Search	Insert	Delete
Array	O(1)	O(n)	O(n)	O(1)
Stack	O(n)	O(n)	O(1)	O(1)
Queue	O(n)	O(n)	O(1)	O(1)
Linked List	O(n)	O(n)	O(1)	O(1)
HashMap / Set	-	O(1)	O(1)	O(1)
Binary Search Tree	O(log n)	O(log n)	O(log n)	O(log n)
Heap (Priority Queue)	-	O(n)	O(log n)	O(log n)

Sorting Algorithms

Algorithm	Best	Average	Worst	Space	Stable
Bubble Sort	O(n)	O(n^2)	O(n^2)	O(1)	Yes
Selection Sort	O(n^2)	O(n^2)	O(n^2)	O(1)	No
Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1)	Yes
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes
Quick Sort	O(n log n)	O(n log n)	O(n^2)	O(log n)	No
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	No
Counting Sort	O(n + k)	O(n + k)	O(n + k)	O(n + k)	Yes
Radix Sort	O(n × k)	O(n × k)	O(n × k)	O(n + k)	Yes

Searching Algorithms

Algorithm	Time Complexity	Use Case
Linear Search	O(n)	Unsorted array
Binary Search	O(log n)	Sorted array
Search in Rotated Array	O(log n)	Rotated sorted array
Search in 2D Matrix	O(log (n × m))	Row and column sorted

Graph Algorithms

Algorithm	Time Complexity	Use Case
BFS	O(V + E)	Shortest path (unweighted)
DFS	O(V + E)	Traversal, cycle detection
Dijkstra	O((V + E) log V)	Shortest path (non negative)
Bellman Ford	O(V × E)	Shortest path (negative weights)
Floyd Warshall	O(V^3)	All pairs shortest path
Kruskal (MST)	O(E log E)	Minimum spanning tree
Prim (MST)	O((V + E) log V)	Minimum spanning tree
Topological Sort	O(V + E)	DAG ordering
Disjoint Set Union	O(a(V))	Union find / Kruskal
SCC (Kosaraju)	O(V + E)	Strongly connected components

@codedsoul_05

Dynamic Programming Patterns

Pattern	Problem Examples	Approach
1D DP	Fibonacci, Climbing Stairs	dp[i] from previous states
2D DP	Grid Paths, LCS, Edit Distance	dp[i][j] with two parameters
Knapscck	0/1 Knapasck, Subset Sum	Take / Not Take
Unbounded	Coin Change, Rod Cutting	Use same item multiple times
DP on Strings	LCS, Edit Distance	Compare prefixes
DP on Trees	Tree DP, Diameter	Post order traversal
DP on Bitmask	TSP, Subset problems	State represented by bits
DP on Intervals	Matrix Chain, Burst Balloons	Solve smaller intervals first

Greedy Algorithms

Problem	Strategy	Example
Activity Selection	Pick earliest finishing	Max non overlapping intervals
Huffman Coding	Pick lowest frequency	Data compression
Fractional Knapasck	Pick highest value/weight	Maximize value
Minimum Spanning Tree	Pick smallest edge	Kruskal / Prim
Dijkstra	Pick smallest distance	Shortest path (non negative)

Common Algorithm Templates

Sliding Window (Fixed Size) – Python

```python
def sliding_window(nums, k):
    window_sum = sum(nums[:k])
    ans = window_sum
    for i in range(k, len(nums)):
        window_sum += nums[i] - nums[i - k]
        ans = max(ans, window_sum)
    return ans
```

Two Pointers

```python
left, right = 0, len(nums) - 1
while left < right:
    if condition(nums[left],
        nums[right]):
        # move pointers
        left += 1
    else:
        right -= 1
```

BFS Template

```python
from collections import deque
def bfs(graph, start):
    visited = set()
    q = deque([start])
    visited.add(start)
    while q:
        node = q.popleft()
        for nei in graph[node]:
            if nei not in visited:
                visited.add(nei)
                q.append(nei)
```

DFS Template

```python
def dfs(graph, node, visited):
    visited.add(node)
    for nei in graph[node]:
        if nei not in visited:
            dfs(graph, nei, visited)
```

Dijkstra Template

```python
import heapq
def dijkstra(graph, src):
    dist = [float('inf')] * n
    dist[src] = 0
    pq = [(0, src)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]: continue
        for v, w in graph[u]:
            if dist[v] > d + w
                dist[v] = d + w
                heapq.heappush(pq, (dist[v], v))
    return dist
```

Bit Manipulation

```python
x & (x - 1)     # remove last set bit
x & -x          # last set bit
x | (1 << i)    # set ith bit
x & ~(1 << i)   # clear ith bit
x ^ (1 << i)    # toggle ith bit
bin(x).count('1')   # count set bits
```

100 Interview Questions (Categorized)

Arrays (10)

1. Two Sum
2. Best Time to Buy and Sell Stock
3. Maximum Subarray
4. Product of Array Except Self
5. Maximum Product Subarray
6. Find Missing Number
7. Find Duplicate Number
8. Merge Intervals
9. Rotate Array
10. Set Matrix Zeroes

Strings (10)

1. Valid Anagram
2. Longest Substring Without Repeating
3. Longest Palindromic Substring
4. Group Anagrams
5. Valid Parentheses
6. Minimum Window Substring
7. String to Integer (atoi)
8. Reverse Words in a String
9. Longest Common Prefix
10. Edit Distance

Linked List (10)

1. Reverse a Linked List
2. Detect Cycle
3. Merge Two Sorted Lists
4. Remove Nth Node from End
5. Reorder List
6. Copy List with Random Pointer
7. Add Two Numbers
8. Find Middle Node
9. Intersection of Two Lists
10. Palindrome Linked List

Two Pointers / Sliding Window (10)

1. Container With Most Water
2. 3Sum
3. Longest Substring Without Repeating
4. Minimum Size Subarray Sum
5. Subarray Product Less Than K
6. Trapping Rain Water
7. Find All Anagrams in a String
8. Max Consecutive Ones III
9. Longest Repeating Character Replacement
10. Fruits Into Baskets

Stack & Queue (10)

1. Valid Parentheses
2. Min Stack
3. Evaluate Reverse Polish Notation
4. Daily Temperatures
5. Sliding Window Maximum
6. Largest Rectangle in Histogram
7. Implement Queee Using Stacks
8. Implement Stack Using Queues
9. Basic Calculator
10. Decode String

Trees (10)

1. Inorder Traversal
2. Validate Binary Search Tree
3. Lowest Common Ancestor
4. Binary Tree Right Side View
5. Maximum Depth of Binary Tree
6. Diameter of Binary Tree
7. Serialize and Deserialize
8. Kth Smallest in BST
9. Construct Tree from Traversals
10. Binary Tree Maximum Path Sum

Graphs (10)

1. Number of Islands
2. Clone Graph
3. Course Schedule
4. Pacific Atlantic Water Flow
5. Network Delay Time
6. Cheapest Flights Within K Stops
7. Redundant Connection
8. Word Ladder
9. Alien Dictionary
10. Minimum Spanning Tree

Dynamic Programming (10)

1. Climbing Stairs
2. House Robber
3. House Robber II
4. 0/1 Knapsack
5. Coin Change
6. Longest Increasing Subsquence
7. Longest Common Subsequence
8. Edit Distance
9. Unique Paths
10. Minimum Path Sum

Greedy + Others (10)

1. Jump Game
2. Jump Game II
3. Partition Labels
4. Merge Intervals
5. Non Overlapping Intervals
6. Meeting Rooms
7. Task Scheduler
8. Gas Station
9. Candy
10. Kth Largest Element in a Stream